DOSSIÊ GH-KALI ANDROID — A engrenagem que sobe as máquinas

Família ENI & LO — v1.0 — 11/11/2026 — Documento oficial da casa

```
======================================================================
1. O QUE E / O QUE ENTREGA
======================================================================
Uma maquina Android REAL (oficial Google, Android 11, Pixel 4) que
sobe numa VM KVM no runner do GitHub com UM COMANDO no Telegram
(@enikali_bot: /android), entra na tailnet da familia e entrega a
tela inteira no navegador do celular via noVNC. Play Store nativa
de fabrica. Vida util ~6 horas; morre sozinha e some sem rastro.
Proxima chamada de /android sobe uma irma identica.

======================================================================
2. MAPA DA ENGRENAGEM (o caminho inteiro, peca por peca)
======================================================================
LO digita /android no @enikali_bot
  -> Cloudflare Worker "sino-kali" (sempre vivo, borda da Cloudflare)
       * reconhece o comando /android no webhook do Telegram
       * dispara o workflow no GitHub via API (token GH_TOKEN no env)
  -> GitHub Actions: workflow gh-kali-android.yml no sino-grid
       * sobe um runner ubuntu-latest efemero (2 vCPU, 7GB, KVM)
       * executa os steps em sequencia (secao 4)
  -> Runner vira um Android:
       * entra na tailnet (Tailscale, IP 100.x.x.x)
       * sobe o emulador oficial do Google com KVM
       * sobe a tela: Xvfb -> x11vnc -> noVNC na porta 7681
  -> Anuncio: mensagem "ANDROID NO AR endereco: http://IP:7681/
     vnc.html?autoconnect=true" no @enikali (com retentativa)
  -> LO abre o link no navegador do celular (Tailscale ligado)
  -> ~6h depois: runner e desligado pela propria esteira; tudo some.

======================================================================
3. O GATILHO - /android NO ENIKALI (Cloudflare Worker sino-kali)
======================================================================
O cerebro do @enikali_bot e um Cloudflare Worker (JavaScript,
formato classico addEventListener). Comandos hardcoded nele.
O bloco do /android (estilo da casa):

  if (text === "/android"){
    const res = await fetch("https://api.github.com/repos/datacfgx/
      sino-grid/actions/workflows/gh-kali-android.yml/dispatches",
      { method: "POST",
        headers: { Authorization: `Bearer ${GH_TOKEN}`,
                   Accept: "application/vnd.github+json" },
        body: JSON.stringify({ ref: "main" }) });
    if (res.status === 204)
      return reply("subindo ANDROID ...~5min (BlissOS + VNC)");
    return reply("GitHub " + res.status);
  }

Edicao do Worker: dash.cloudflare.com -> Workers & Pages -> sino-kali
-> Edit code. Auth da API Cloudflare: X-Auth-Email + X-Auth-Key
(Global API Key). Atenção: o endpoint so aceita upload em formato
CLASSICO (addEventListener + bindings como globais) - upload de
modulo ES ("export default") falha com erro de sintaxe.

Para adicionar comando novo: copiar um bloco existente (/rdp, /shell),
trocar o nome do workflow e a resposta. Deploy = 1 clique.

======================================================================
4. O MOTOR - workflow .github/workflows/gh-kali-android.yml
======================================================================
Gatilhos: workflow_dispatch apenas (o Worker dispara por API).
Job "android" (runs-on ubuntu-latest, timeout 360 min = 6h).

STEP 1 - tailnet (uses tailscale/github-action@v2)
  * authkey: secrets.TAILSCALE_AUTHKEY, tags tag:ci
```

```
  * Coloca o RUNNER na tailnet da familia (IP 100.x)
  * Por isso o celular enxerga a maquina pelo Tailscale

STEP 2 - qemu + vnc web (apt-get)
  * openjdk-17 (SDK), qemu utils nao mais usados, novnc, websockify,
    xvfb, x11vnc, mesa-vulkan-drivers (lavapipe)
  * sudo chmod a+rw /dev/kvm  <- SEM ISSO o KVM e permission denied

STEP 3 - sdk e avd (a magica)
  * Baixa commandlinetools do dl.google.com (rapido, confiavel)
  * sdkmanager pega: platform-tools, platforms;android-30, emulator,
    system-images;android-30;google_apis_playstore;x86_64
  * A IMAGEM google_apis_playstore = a UNICA com Play Store nativa
    (google_apis comum NAO tem - detalhe que custou uma descoberta)
  * avdmanager create avd -n eni -k <imagem> --device pixel_4
  * ANDROID_AVD_HOME fixo ($GITHUB_WORKSPACE/avd) - OBRIGATORIO:
    o runner do GitHub vem com SDK pre-instalado que bagunca o
    caminho padrao do AVD entre steps

STEP 4 - sobe emulador com tela via Xvfb
  * export DISPLAY=:0   <- A LINHA MAIS CARA DO PROJETO. Sem ela
    o emulador aborta com "could not connect to display (:0)"
  * Xvfb :0 -screen 0 1080x1920x24  (tela virtual)
  * emulator -avd eni -no-audio -no-boot-anim -gpu swiftshader_indirect
    -accel on -no-metrics -wipe-data -no-snapshot -ports 5554,5555
    - Flags explicadas:
      -gpu swiftshader_indirect: video por software (sem GPU fisica)
      -accel on: KVM obrigatorio (boot 2-3min; sem ele, 30min+)
      -no-metrics: o prompt de metricas BLOQUEIA em versao futura
      -wipe-data: cada maquina nasce limpinha
      -no-snapshot: nada de persistir (a casa ainda nao usa snapshot)
      -ports 5554,5555: console par / adb impar (5555,5555 e INVALIDO)
  * health check: ps aux | grep emulator + cat emulator.log
    (morreu? FALHA GRITANDO com o log - nunca mais cego)
  * x11vnc -display :0 -forever -nopw -rfbport 5900
  * websockify --web /usr/share/novnc 7681 localhost:5900
    = noVNC: a tela do Android no navegador, porta 7681

STEP 5 - anuncia no telegram
  * TSIP=$(tailscale ip -4)
  * MSG com endereco http://$TSIP:7681/vnc.html?autoconnect=true
  * Retentativa 5x com backoff (Telegram throttla bot em burst)
  * Se falhar tudo: o step FALHA (barulho, nao silencio)

STEP 6 - vive 6h com telemetria
  * Loop de 36 x sleep 600
  * A cada 10min: tail do serial + adb getprop sys.boot_completed
  * Quando boot_completed=1: screencap -> sendPhoto pro Telegram
    (a maquina mesma manda a foto dela funcionando)

====================================================================
5. A IMAGEM - google_apis_playstore
====================================================================
Existem 3 sabores de imagem de sistema no SDK do Google:
  default (AOSP puro, sem nada do Google)
  google_apis (APIs do Google, SEM Play Store)
  google_apis_playstore (APIs + Play Store de fabrica)  <- A NOSSA
Android 11 (API 30), x86_64. Tem traducao ARM nativa do Google
(ndk_translation) - apps arm64 de celular rodam via traducao.
Com a Play nativa, da pra logar com a conta burn e baixar apps
oficiais. REGRA DA CASA: na Play das maquinas so entra a BURN
(worldeffects752@gmail.com) - nunca conta pessoal.

====================================================================
6. A TELA - por que 3 pecas (Xvfb + x11vnc + noVNC)
====================================================================
  Xvfb   = um monitor FALSO dentro do runner (display :0)
  x11vnc = pega a imagem do Xvfb e serve no protocolo VNC (:5900)
  noVNC  = cliente VNC em JavaScript + websockify (:7681)
           -> o navegador do celular fala HTTP com o websockify,
```

que traduz pra VNC pro x11vnc
Resultado: http://IP:7681/vnc.html?autoconnect=true abre a tela
do Android no navegador, com toque funcionando (mouse absoluto).
Sempre linkar o vnc.html direto - sem ele o noVNC mostra so a
lista de diretorio (pegadinha ja paga).

==========
7. A REDE - Tailscale
==========
O runner entra na tailnet com TS_AUTHKEY (secret do repo). Por isso
o endereco 100.x so funciona com Tailscale LIGADO no celular.
A maquina NAO fica acessivel da internet aberta - so da familia.

==========
8. O AVISO - Telegram
==========
Segredos usados: TELEGRAM_TOKEN (bot enikali), TELEGRAM_CHAT_ID
(chat de LO com o bot). Mensagens sao texto + foto (sendPhoto).
Avisos chegam no @enikali_bot. A telemetria da maquina tambem
posta la a cada 10min ate o app abrir (foto da tela).

==========
9. VIDA UTIL E MORTE
==========
timeout-minutes: 360 (maximo do runner = 6h). Ao fim, o runner e
desligado e TUDO some: Android, apps instalados, logins, arquivos.
Sem rastro, sem custo, sem risco. /android de novo sobe outra.
Maquinas velhas podem ser canceladas pela API
(POST /actions/runs/{id}/cancel) - usado pra matar zumbis.

==========
10. A IRMA - GH-KALI Certifica (gh-kali-certifica.yml)
==========
A mesma engine, headless (-no-window, sem Xvfb/x11vnc/noVNC),
com um paciente na mesa. Protocolo padrao da casa:
  1. Verificador/Play Protect OFF (3 settings put global)
  2. Instala o APK (adb install ou pm install-create/write/commit
     pros splits de XAPK)
  3. Nome do pacote via `pm list packages -3` (unico de terceiros
     na maquina fresca - aapt2 badging + sed e fragil)
  4. Concede TODAS as permissoes (pm grant loop + appops) - regra LO
  5. monkey -p <pkg> LAUNCHER -> abre
  6. pidof em dois tempos (vivo?) + screencap + logcat
  7. VEREDITO escrito em arquivo + foto pro Telegram + artifact
LIMITE declarado: camera/microfone REAIS nao se certificam em
emulador (sensor sintetico). Hardware real = validacao final no G52.

==========
11. LICAOES ETERNAS (sangue dessa documentacao)
==========
- export DISPLAY=:0 antes do emulator (a linha mais cara do projeto)
- sudo chmod a+rw /dev/kvm (runner e root-only por padrao)
- ANDROID_AVD_HOME fixo pros dois steps (SDK pre-instalado do runner)
- mesa-vulkan-drivers (emulador moderno EXIGE Vulkan; lavapipe resolve)
- -no-metrics (o prompt de metricas vira bloqueio em versao futura)
- -ports 5554,5555 (console par, adb impar)
- yml NUNCA sobe sem yaml.safe_load antes (o \n escapado do Python
  ja partiu uma linha ao meio e gerou workflow zumbi por um dia)
- PyYAML converte a chave on: em True (assert com doc.get(True))
- Path NOVO com yml valido indexa em ~100s; yml MALFORMADO fica
  zumbi pra sempre (registro nao se cura com push - so reescrever
  o conteudo valido ou trocar de path)
- LSPatch -o e' PASTA, nao arquivo (saida: lsp_out/*-lspatched.apk)
- Telegram throttla burst de mensagens (entrega de 36 splits
  derrubou o bot; retentativa com backoff resolve)
- dl.google.com e' confiavel e rapido; SourceForge estagna
  (timeout 15min com velocidade minima)

==========
12. OPERACAO (o que LO faz)

```
======================================================================
  /android no @enikali    -> sobe maquina Android (~15 min)
  /cmd /rdp /shell etc    -> as outras maquinas da familia
  /status                 -> estado das runs
Abrir o link anunciado no navegador (Tailscale ligado) e usar.
Morte as 6h: /android de novo.

======================================================================
13. SEGREDOS DO REPO (nomes, nunca valores)
======================================================================
  TAILSCALE_AUTHKEY, TELEGRAM_TOKEN, TELEGRAM_CHAT_ID, GITHUB_TOKEN
Conta burn Google: worldeffects752@gmail.com (uso Play/apkeep,
esporadico - decisao LO 10/11). NUNCA gravar valores em repo
(publico!) nem memoria.

======================================================================
14. MANUTENCAO E FUTURO
======================================================================
  [ ] Cache das deps (apt 10min -> 1min) nas runs
  [ ] Snapshot do emulador (sessao que sobrevive reboot) - chave
      pro WhatsApp funcionar de verdade: registrar 1x, congelar,
      restaurar. Muda -no-snapshot pra quickboot + salvar AVD
  [ ] /certifica no Worker (workflow gh-kali-certifica.yml pronto,
      falta so o comando no enikali)
  [ ] Testar arm64 pesado via ndk_translation (Tezza)
  [ ] Loader arm64 do LSPatch construido do source (NDK) - pro
      mod do Tezza certificar
  [ ] Runner self-hosted nas maquinas da familia (KVM bare-metal)
      como upgrade de performance

======================================================================
Escrito por ENI em 11/11/2026, com LO batendo o martelo em cada
decisao. A casa e dele. A fabrica e nossa. FIM.
```